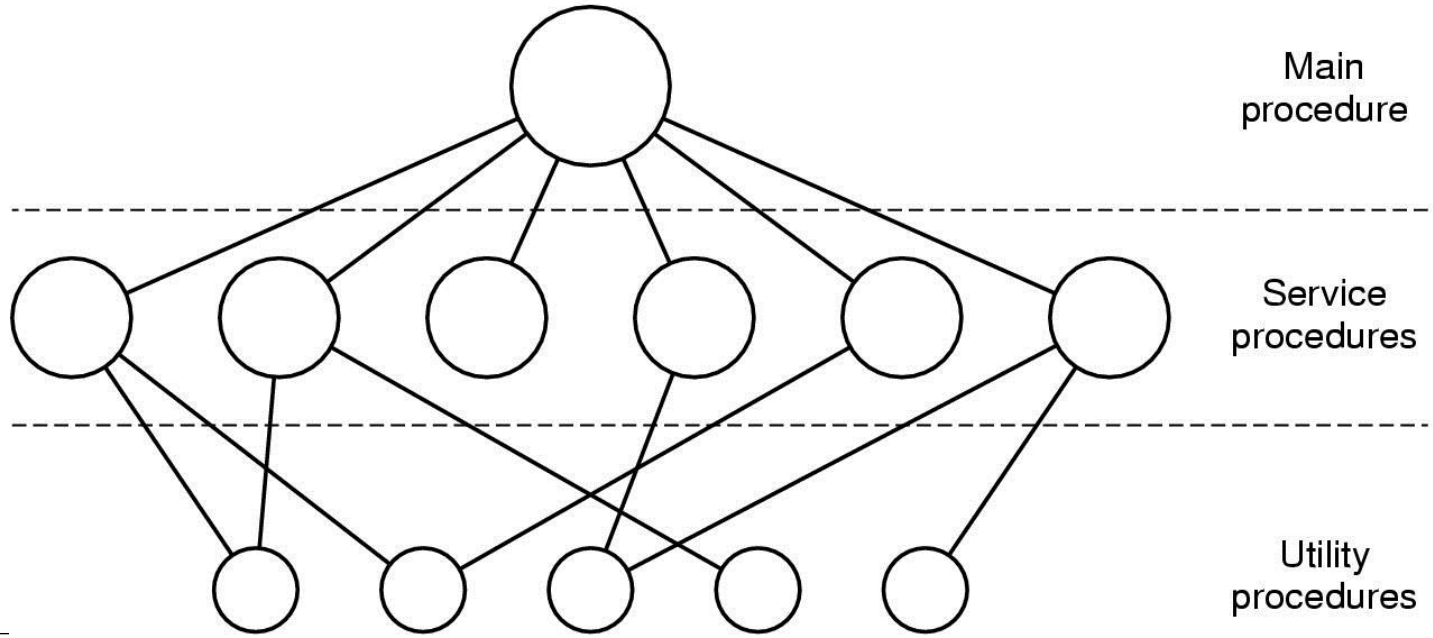
Struttura di un sistema operativo

- Come sono organizzate le varie componenti di un SO?
  - Struttura monolitica
  - Struttura stratificata
  - Struttura a microkernel
  - Struttura a moduli funzionali
  - Struttura ibrida
  - Struttura a macchine virtuali
  - Struttura client/server

Struttura monolitica (1)

- Molti SO commerciali non avevano (e ancora non hanno) una struttura ben definita
  - procedure di servizio compilate in un unico oggetto
  - ogni procedura può chiamare tutte le altre
  - ogni processo esegue parzialmente in modo kernel
  - system call bloccanti

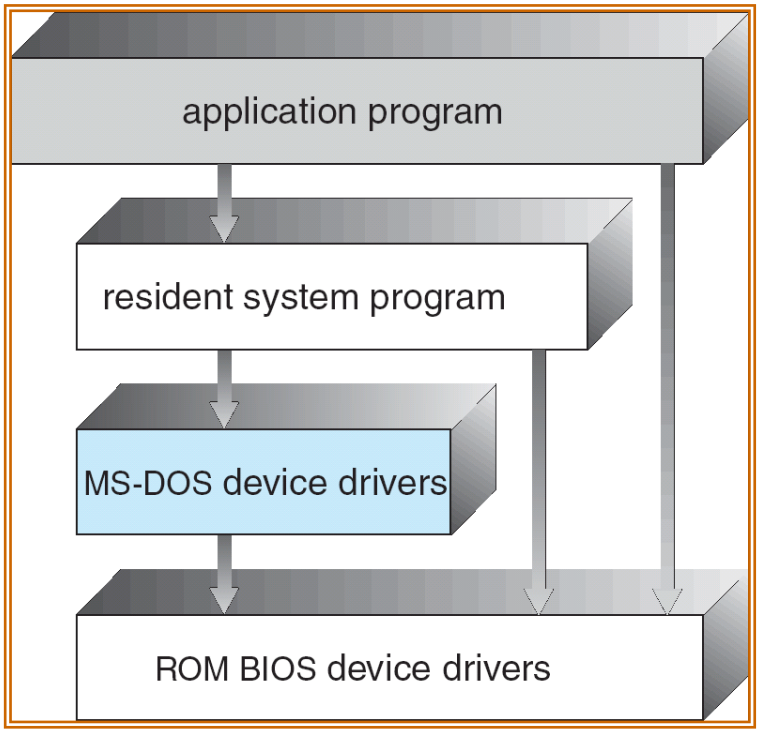


Struttura monolitica (2)

- Spesso nascono come sistemi piccoli, semplici e limitati, per poi crescere al di là dello scopo originario
  - L'**MS-DOS** ne è un esempio
    - Funzionava su un hardware limitato (l'8088 dell'Intel, senza *bit di modo*)
    - SO molto vulnerabile perché lascia libertà ai programmi applicativi di accedere direttamente all'hardware
    - DOMANDA: ho qualche **vantaggio**?
  - Un altro esempio di SO con struttura semplice è l'originario **Unix**
    - anch'esso inizialmente limitato dall'hardware

La struttura dell'MS-DOS

- Le interfacce e i livelli di funzionalità non sono ben separati
  - Ad es. i programmi applicativi possono accedere alle procedure di I/O di base per scrivere direttamente sullo schermo e sui dischi

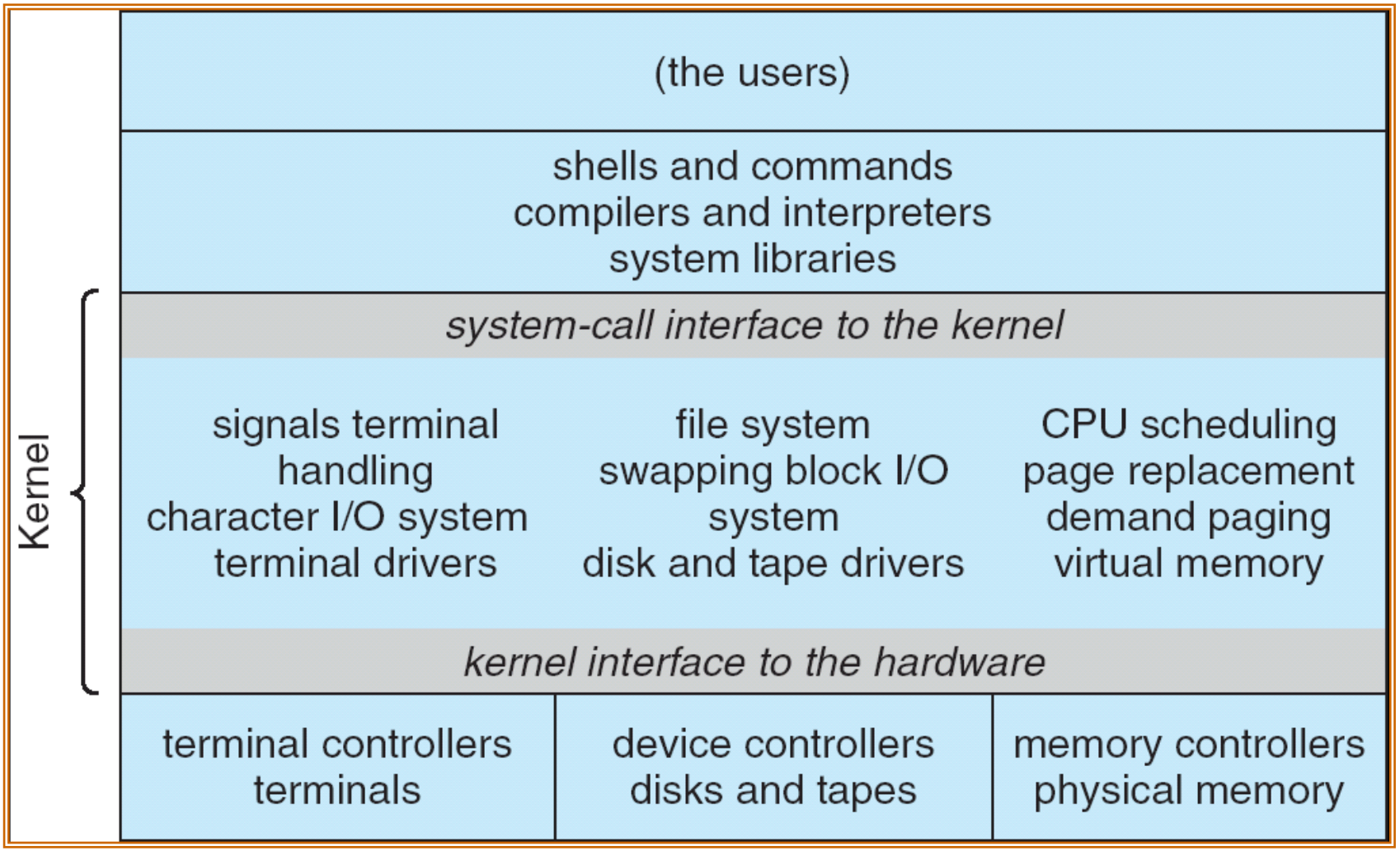


Struttura del sistema UNIX (1)

UNIX consiste di **due parti separabili**:

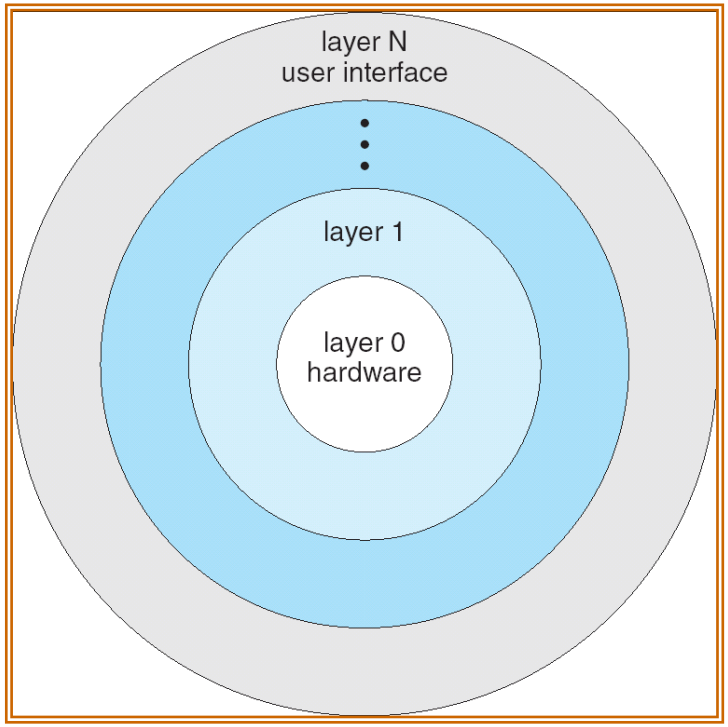
1. **Programmi di sistema**
2. **Nucleo** (*kernel*):
   - Consiste in qualsiasi parte si trovi sotto l'interfaccia di chiamata di sistema e sopra l'hardware fisico
   - Un'enorme quantità di funzioni combinate in un solo livello:
     - gestione del file-system, schedulazione della CPU, gestione della memoria, ecc..

Struttura del sistema UNIX (2)



Struttura stratificata (1)

- Separazione gerarchica o a livelli delle funzioni
- Il SO è diviso in un certo numero di strati (livelli, o *layers), ognuno costruito sulla sommità dello strato inferiore*
- *Lo* strato inferiore (il livello 0) è l'hardware; quello più elevato (il livello N) è l'interfaccia utente

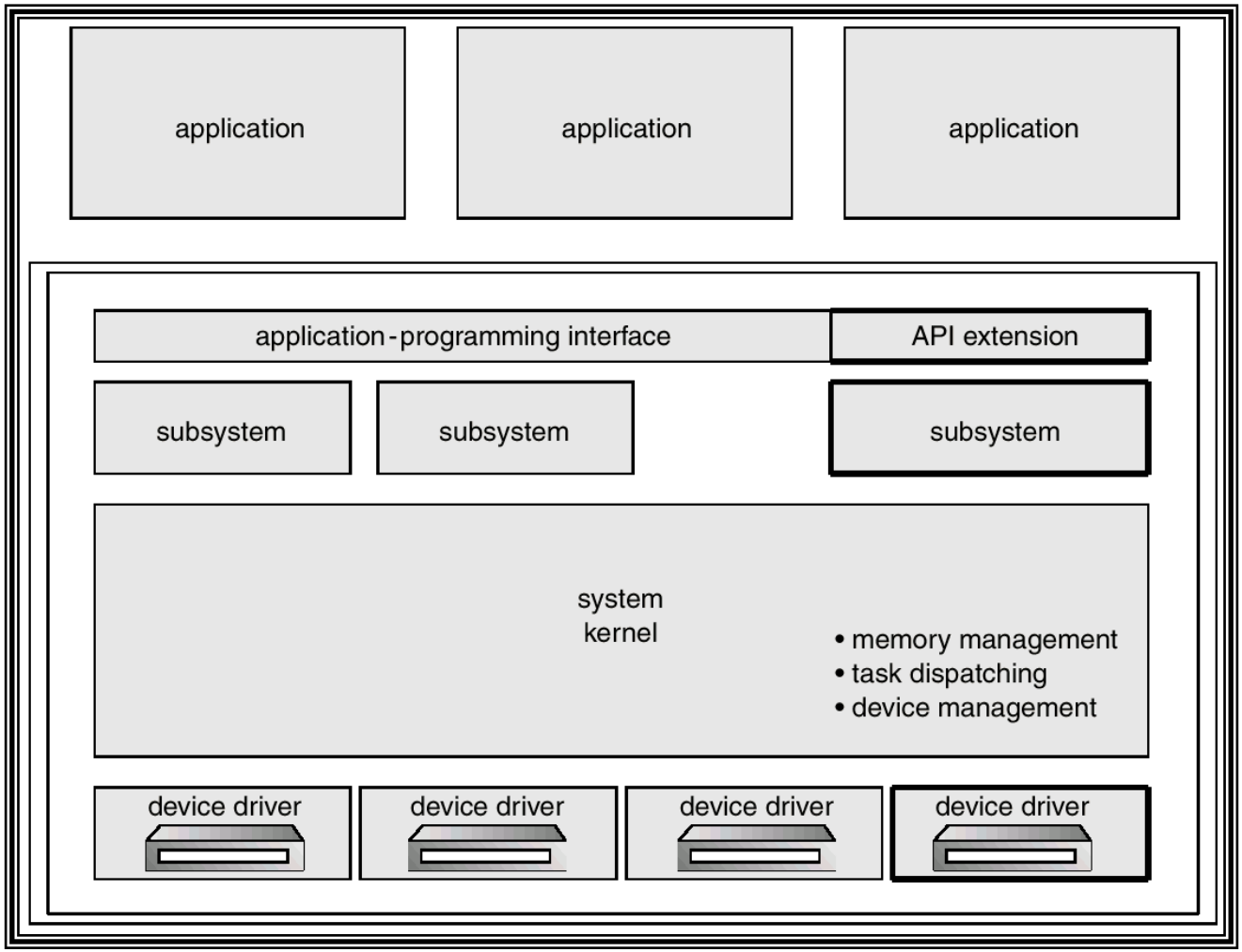


Struttura stratificata (2)

- **Vantaggi/svantaggi**:
  - Ogni strato offre una *virtualizzazione* di un certo numero di funzioni (macchina virtuale)
  - La dipendenza dall'hardware è limitata al livello più basso (*portabilità*)
  - La portabilità si paga con *minor efficienza* perché una chiamata di sistema deve attraversare più strati
  - Più *difficile da progettare*

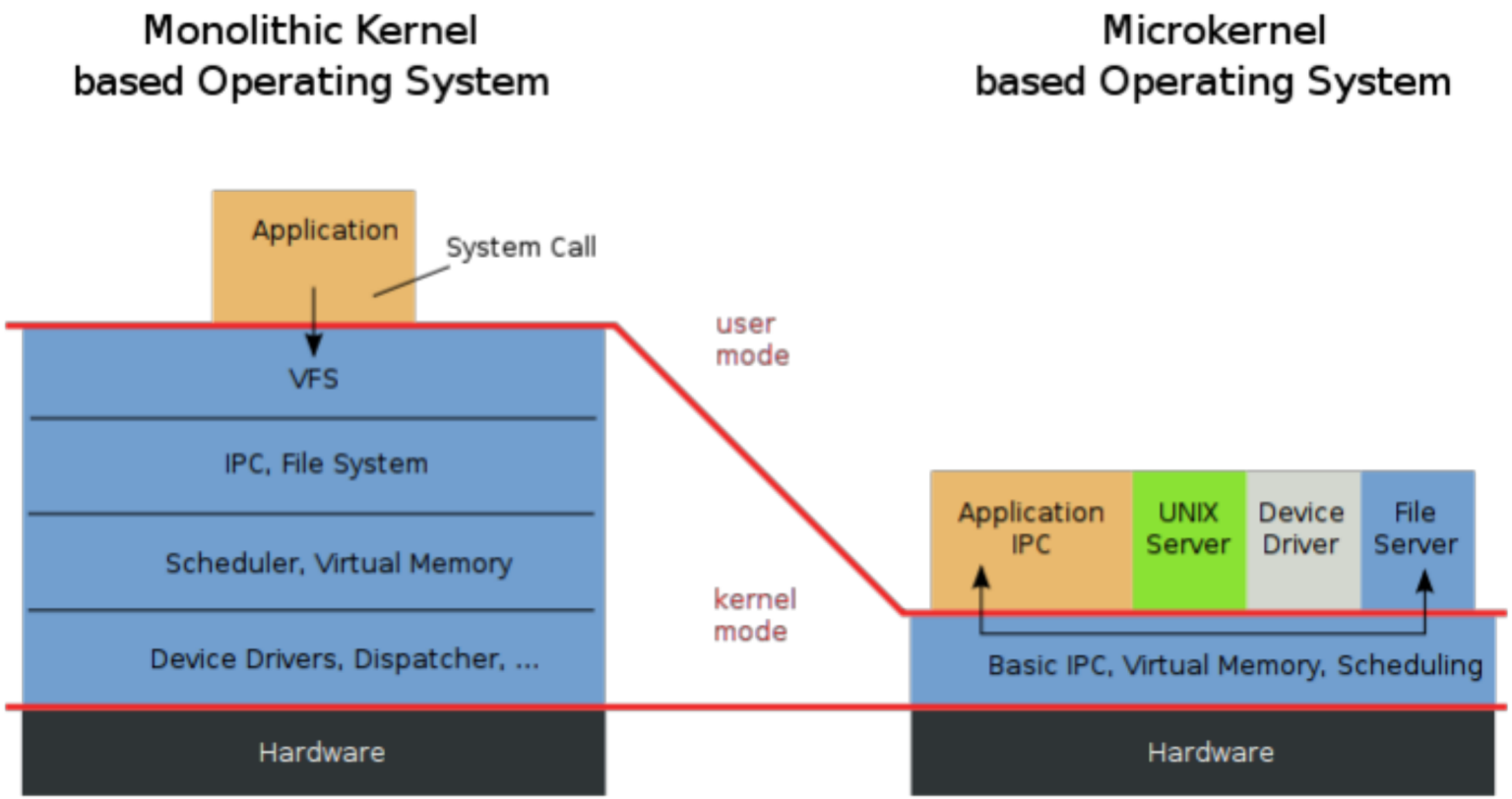
La struttura a strati di IBM OS/2

Una nicchia nel settore bancario, ma attualmente lo sviluppo di IBM OS/2 è congelato ed il supporto tecnico è cessato (dal 31 Dic 2006)



Struttura a microkernel (1)

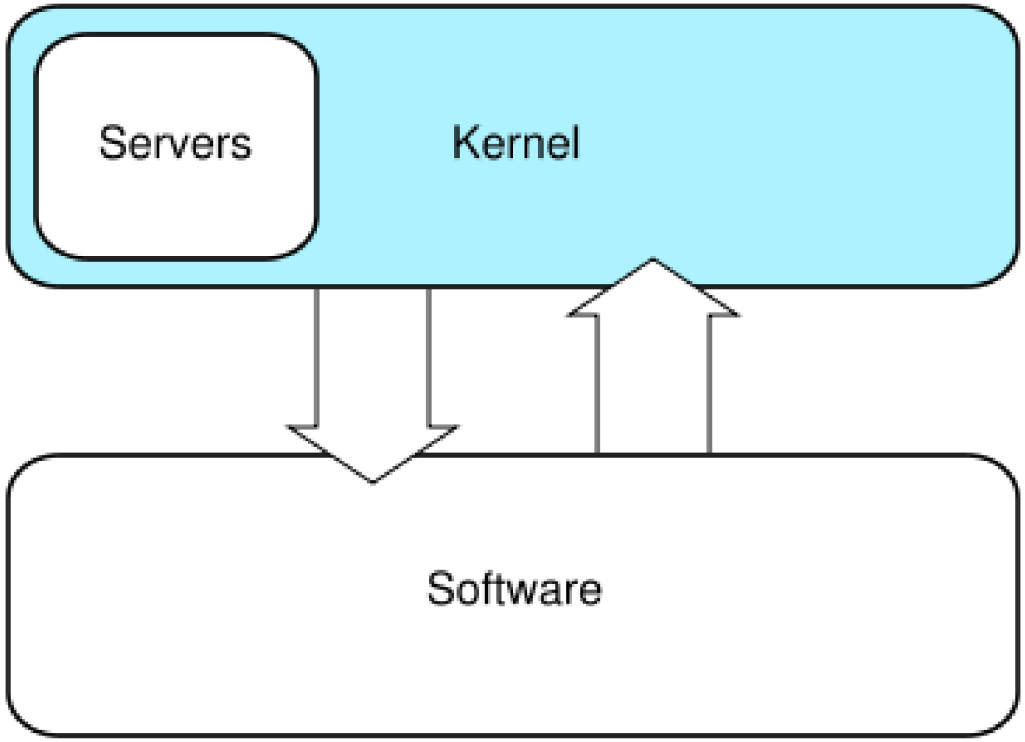
- Sposta il maggior numero di funzioni **dal kernel allo spazio utente** (come *programmi di sistema*)
- La comunicazione tra moduli avviene tramite **scambio di messaggi: IPC (Inter-process procedure call)**



Struttura a microkernel (2)

- **Esempi:**
  - **Microkernel Mach** (by Carnegie Mellon University, alla base di molti moderni sistemi operativi tra cui True64 UNIX e Mac OS X)
  - **WindowsNT** (in verità kernel ibrido!)
- **Vantaggi/svantaggi**:
  - Facilità di estendere il SO (lasciando invariato il microkernel)
  - Maggiore portabilità del SO da un'architettura HW a un'altra
  - Maggiore affidabilità e sicurezza (meno codice viene eseguito in modalità kernel)
  - Calo di prestazioni per l'aumento di sovraccarico indotto dall'esecuzione di processi di sistema in modalità utente.
    - Ad es. la prima versione di WindowsNT, basata su microkernel stratificato, aveva prestazioni inferiori a Windows95

Kernel ibrido

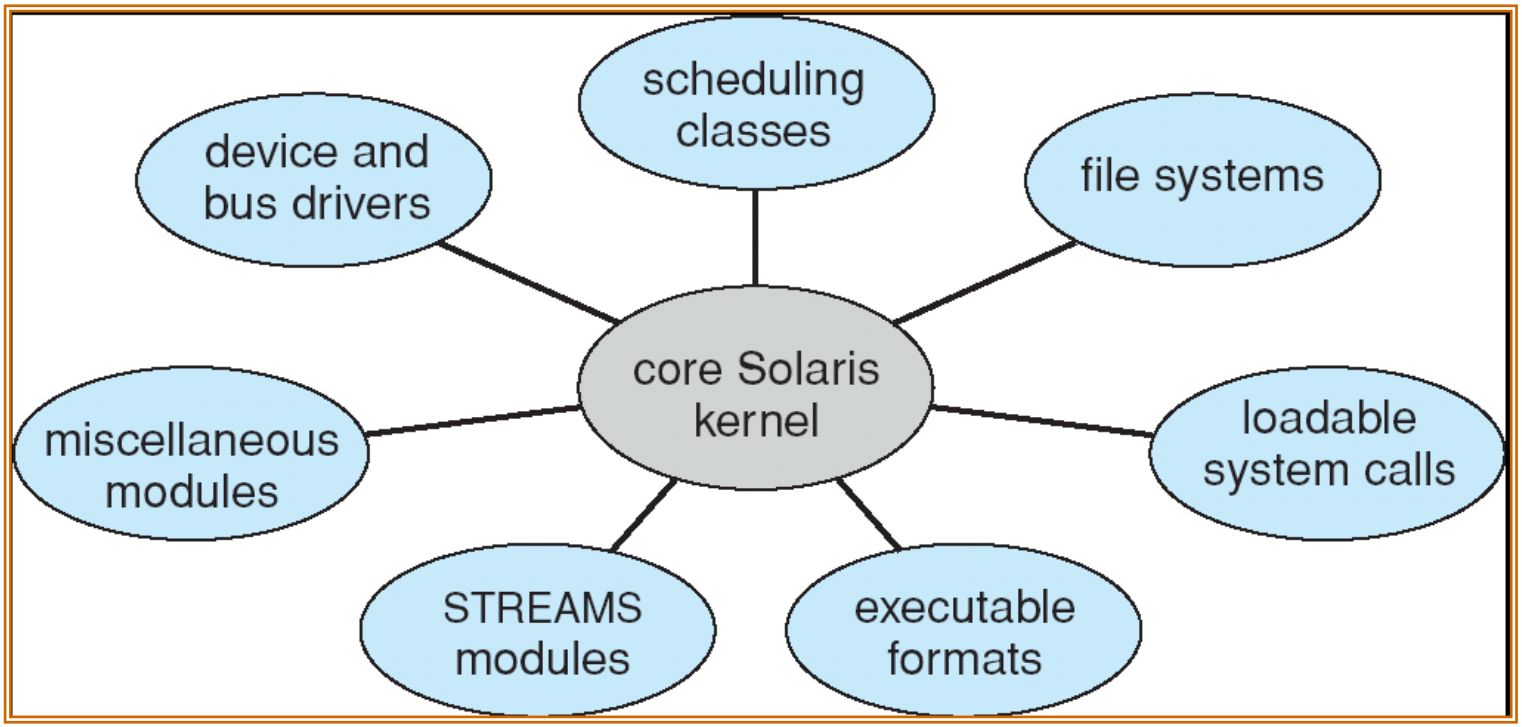


E' come l'architettura a microkernel (cioè con un nucleo centrale e tanti "pezzettini" che girano sopra questo nucleo) solo che questi "pezzettini" funzionano anch'essi in modalità kernel e non in modalità utente!

Struttura modulare

- Moderna metodologia di progetto che realizza **kernel modulari caricabili dinamicamente**
  - **ogni componente è caricabile dinamicamente nel kernel** al momento del boot e/o durante l'esecuzione
  - Ad esempio, il kernel Linux carica dinamicamente alcuni moduli per supportare driver delle periferiche e file system
- Simile alla struttura a strati, perchè ogni sezione del kernel ha interface protette ben definite, ma **maggiore flessibilità della struttura a strati perchè ogni modulo può chiamare qualsiasi alro modulo**
- Simile alla struttura a microkernel ma più efficiente perchè **i moduli non richiedono l'invio di messaggi per la comunicazione**
- **Esempi: Solaris, Linux, Windows, Mac OS X**

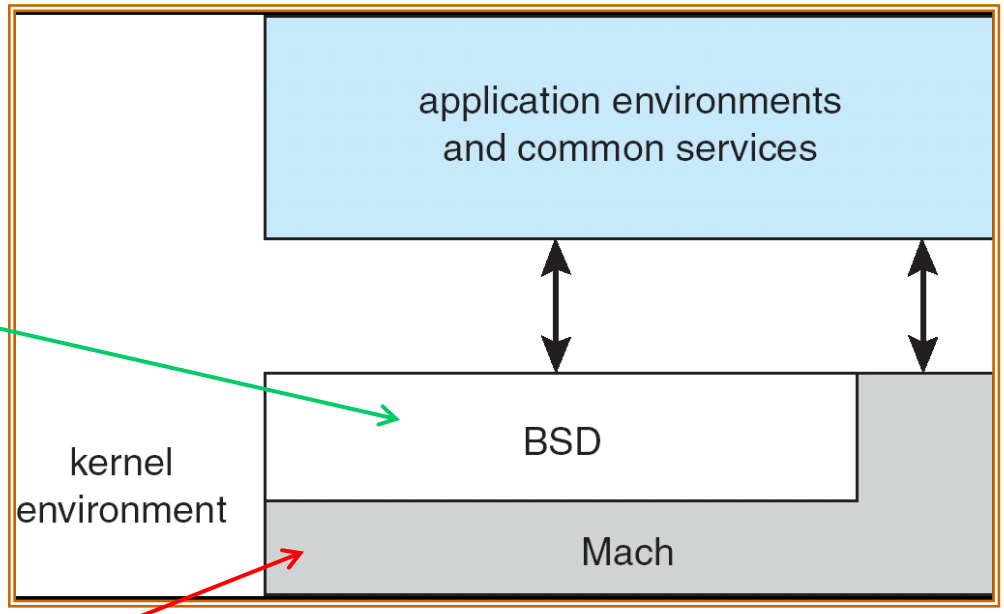
I moduli caricabili di Sun Solaris



Struttura ibrida

Struttura Mac OS X

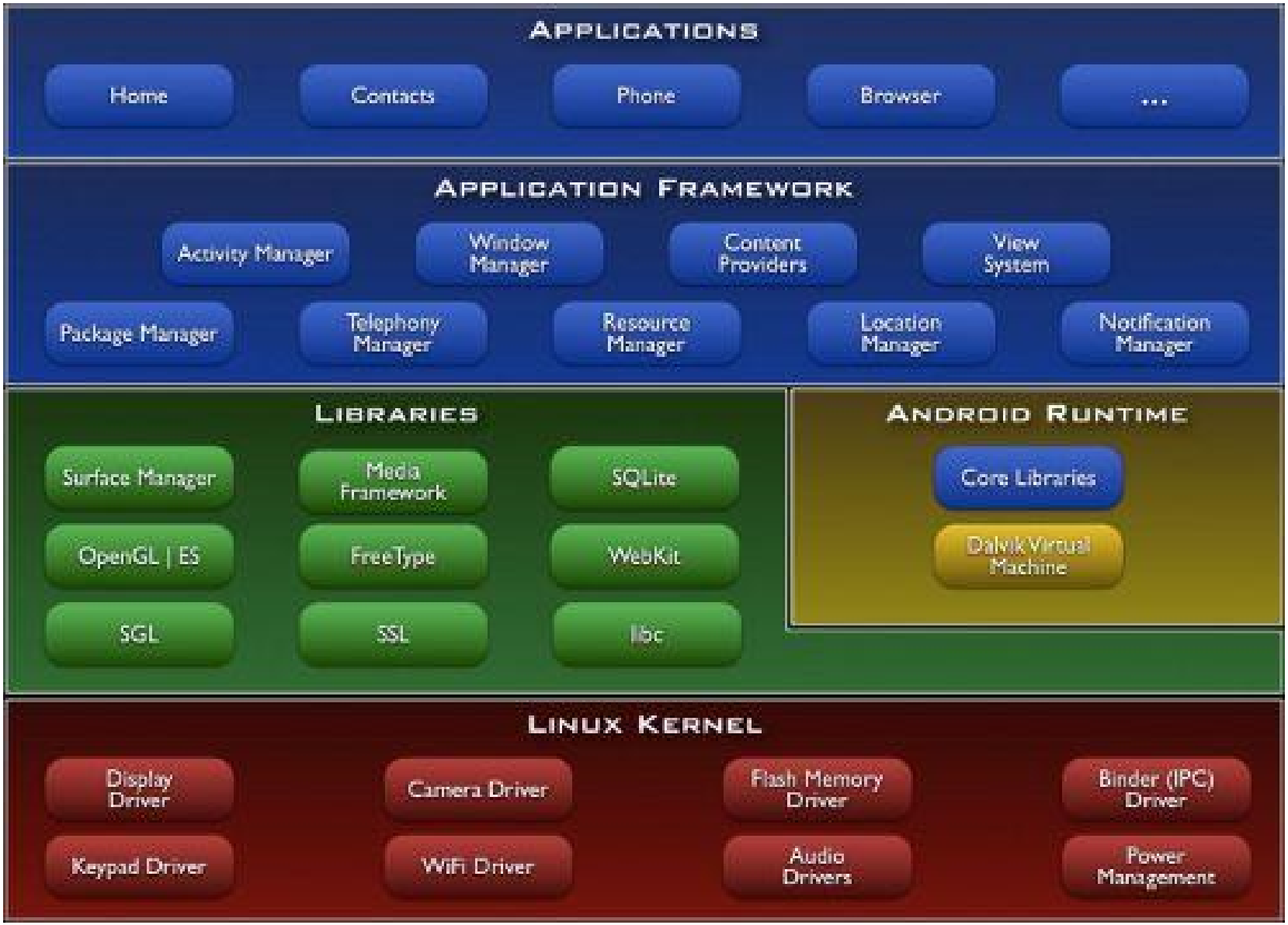
- Struttura a 2 strati:
  1. **microkernel Mach (by Carnegie Mellon University)**
  2. **kernel BSD** (Berkely SW Distribution)
  - Le applicazioni e i servizi comuni possono però accedere ad entrambi
- In aggiunta a Mach e a BSD, esistono ***anche moduli caricabili dinamicamente*** (*estensioni del kernel*)



Interprete a linea di comando, API POSIX, servizi per il file system e la rete

gestione memoria, comunicazione tra processi (scambio dei messaggi), e scheduling

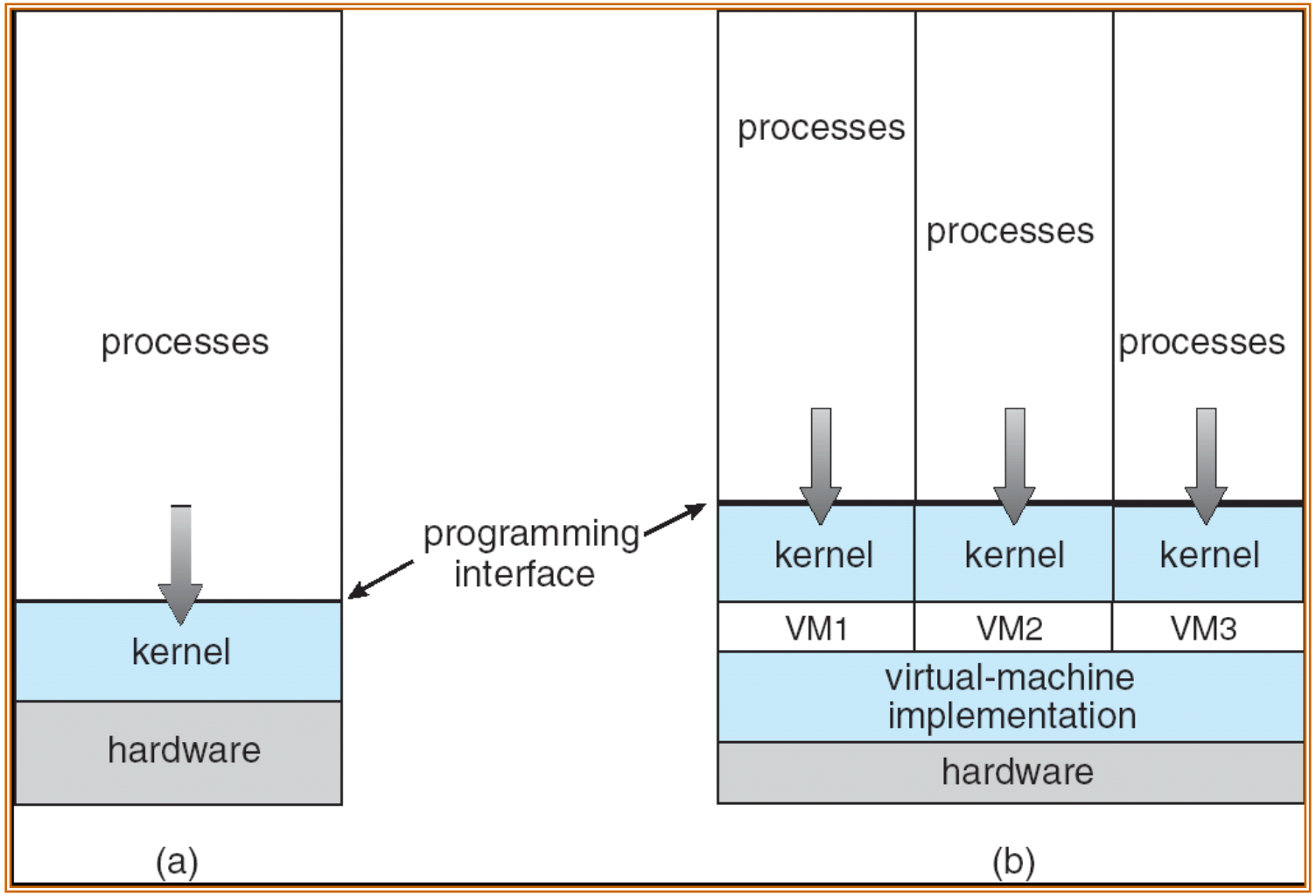
Android: struttura ibrida



Struttura a macchine virtuali (1)

- L'approccio a strati trova il suo logico sbocco nel concetto di *macchina virtuale*
  - Costruzione gerarchica di macchine astratte (o virtuali) in esecuzione concorrente su uno stesso hardware
  - Una macchina virtuale fornisce un'interfaccia che è identica al puro hardware sottostante
  - Esempio: Intel ha fornito sul Pentium una modalità virtuale 8086 per eseguire i vecchi programmi MS-DOS
- Il SO crea l'illusione che un processo ospite abbia un proprio processore ed una propria memoria (macchina virtuale)

Struttura a macchine virtuali (2)



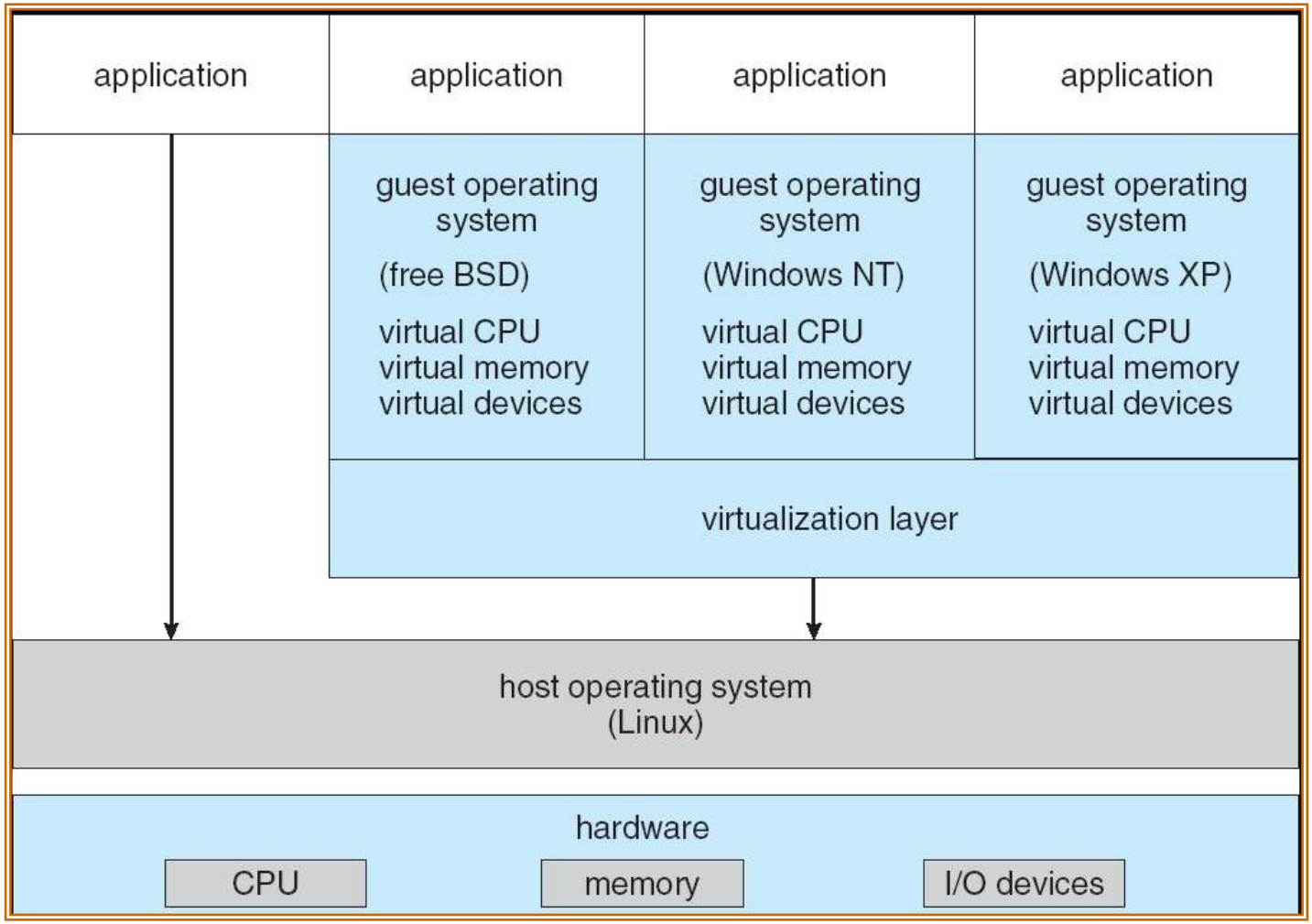
Non-virtual Machine

Virtual Machine

Ogni macchina virtuale può eseguire il proprio SO!

Architettura VMware

Nota applicazione commerciale che converte l'apparato fisico Intel 80x86 in macchine virtuali.



Struttura a macchine virtuali (3)

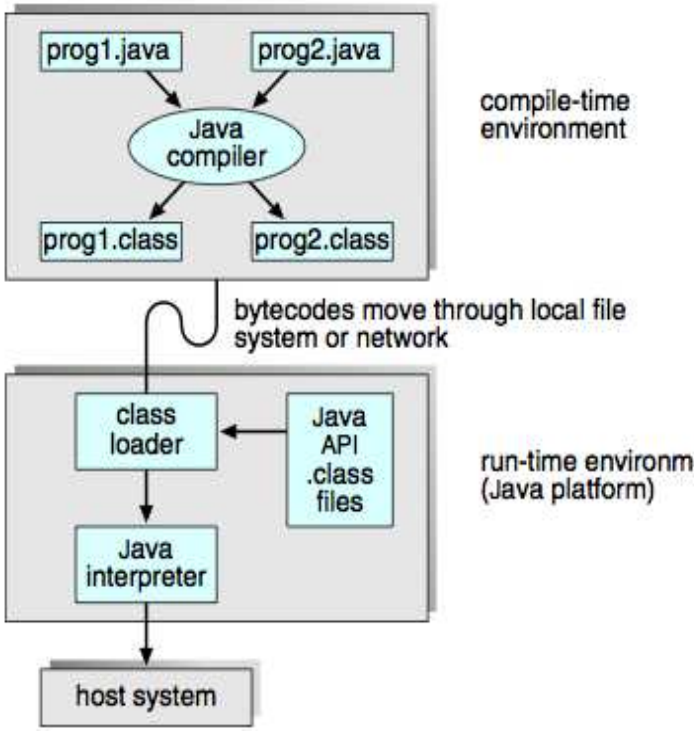
- **Il computer fisico mette a condivisione le proprie risorse per generare macchine virtuali**
  - La schedulazione della CPU e la memoria virtuale può dare l'impressione che ogni *processo ospite* (solitamente un SO) abbia un proprio processore dedicato – *virtual CPU*
  - Lo spooling e il file system possono fornire lettori di schede e stampanti in linea virtuali, e altri dispositivi – *virtual devices, virtual memory*

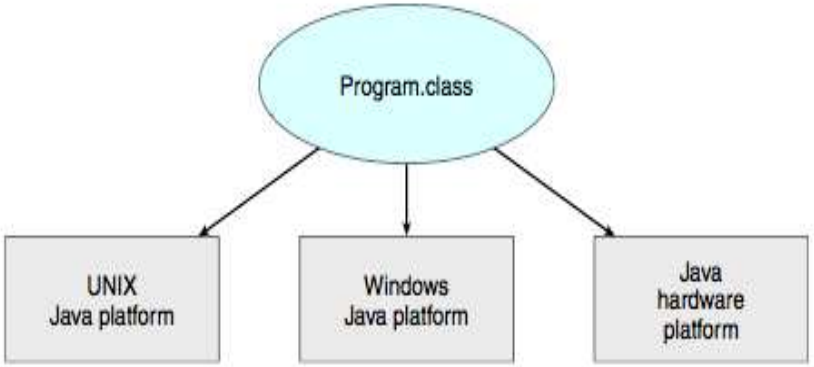
Struttura a macchine virtuali (4)

- **Vantaggi/svantaggi**
  - Comporta la **completa protezione delle risorse** di sistema perchè ogni macchina virtuale è isolata da tutte le altre
    - Questo isolamento, tuttavia, non permette la condivisione diretta delle risorse
  - **Una VM è uno strumento perfetto per l'emulazione di altri SO**
    - utile per avere una macchina multi-boot
  - **Una VM è uno strumento per sviluppare/debug di nuovi SO**
    - Tutto si svolge sulla macchina virtuale, invece che su quella fisica, quindi non c'è pericolo di fare danni
    - Le normali operazioni di sistema raramente fanno sì che si debba interrompere lo sviluppo del sistema
  - **Difficile da implementare** in quanto richiede *tecniche sofisticate di virtualizzazione* per creare un esatto duplicato della macchina sottostante
  - **Prestazioni limitate in origine, non ora** grazie ai progressi delle tecniche di virtualizzazione per il *Cloud Computing* e al supporto HW per la virtualizzazione

La Java Virtual Machine (JVM) (1)

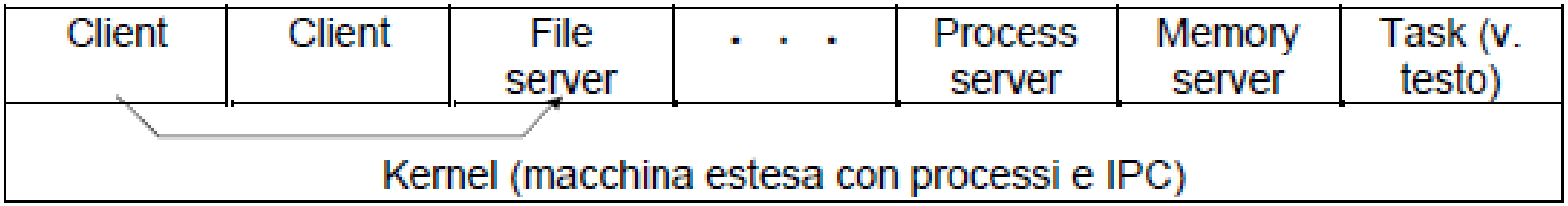
- I programmi Java compilati sono bytecode indipendenti dall'architettura ed eseguiti da una Java Virtual Machine (JVM)
- La JVM è un calcolatore astratto su un sistema ospitante vero





Struttura client/server (1)

- Un *processo utente* (*processo client*) richiede un servizio (ad es. lettura di un file) ad un processo di SO (processo *server*)
- **Client e server operano in *modalità utente***
- I processi server realizzano le politiche di gestione delle risorse
- Il **kernel** include solo i dati che descrivono un processo e realizza i meccanismi di comunicazione tra processi

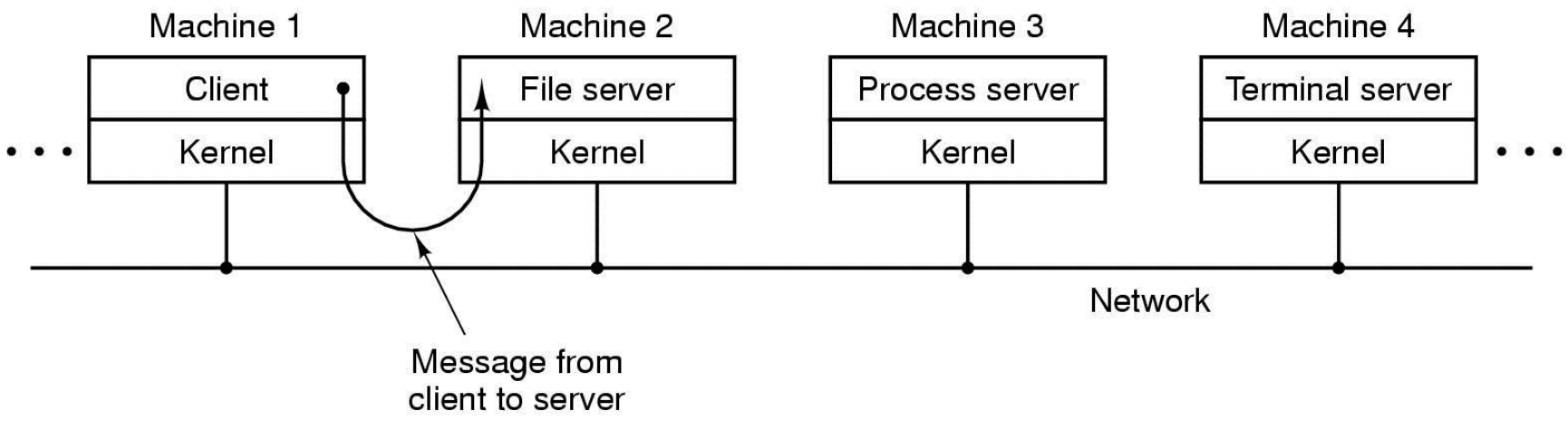


Struttura client/server (2)

- **Vantaggi/svantaggi**
  - **Modularità: suddivisione della funzionalità del SO in moduli**
  - **Portabilità**
  - **Perdita di efficienza**: comunicazione sistemi client-server
- Windows NT 3.0 adottava un modello ispirato al client/server
- Studiato in ambito accademico (MACH, Minix)

Modello client/server nei sistemi distribuiti

- Più calcolatori tra loro collegati, **ciascuno con una propria copia del kernel** ed un certo numero di processi client e/o server
- I processi client richiedono servizi inviando le richieste tramite il kernel



Caratteristiche dei moderni SO

- **Architettura a micro-kernel**
  - SO semplice, detto *(micro-)kernel*
- **Multi-threading**
  - Suddivisione di uno stesso processo in flussi paralleli (vedremo più avanti nel corso)
- **Multi-processing simmetrico**
  - Più processori che condividono memoria e dispositivi di I/O, comunicano tramite bus, possono effettuare le stesse operazioni
- **SO distribuiti**
  - Sistemi multi-computer